Pipeline execution comparison

Thursday, February 12, 2026 4:28 PM

```
for(i=0; i<32; i++)
  C[i] = A[i] + B[i]
```

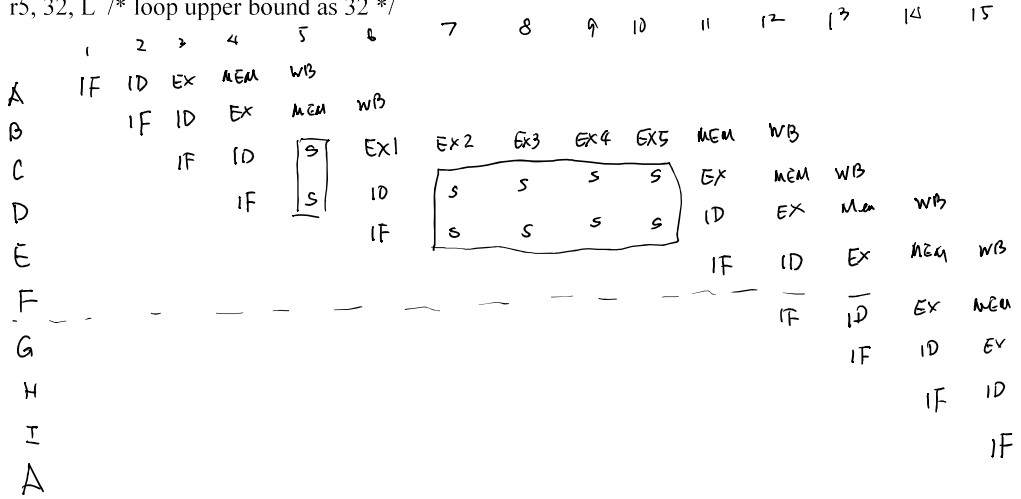
⇒

```
for(i=0; i<32; i+=2)
{ C[i] = A[i] + B[i];
  C[i+1] = A[i+1] + B[i+1];
}
```

```
      MOV   r5, #0
A  L: LOAD  f1, 0(r1)
B     LOAD  f2, 0(r2)
C     FADD  f3, f1, f2
D     STORE f3, 0(r3)
E     ADD   r1, r1, #4
F     ADD   r2, r2, #4
G     ADD   r3, r3, #4
H     ADD   r5, r5, 1
I     BRLE  r5, 32, L  /* loop upper bound as 32 */
```

In-order scalar pipeline

- FADD: 5 cycles, pipelined FAdder.
- Perfect BP.
- Bypass



14 × 32 = --

↓ scheduling

9 × 32

↓ unroll 2x (reduce # of brs, adds at the cost of more registers)

32x

```
LD
LD
ADD
ST
```

```
C[0] = A[0] + B[0];
C[1] = A[1] + B[1];
   :
C[31] = A[31] + B[31];
```

C[0..31] = A[0..31] + B[0..31]

↪ # cycles = 4 × 32 = 128 cycles. (128 insns)

SIMT pipeline

```
id = threadIdx.x;        (r0)
C[tid] = A[tid] + B[tid];   // r1 ⇐ A, r2 ⇐ B, r3 ⇐ C
```

① Configuration: 1 FP unit, 1 ALU. FADD 5 cycle. Warp size 32. 1 SM.

```
A   IMAD  r1, r1, r0, #4   // r1 = r1 + 4 * r0
B   IMAD  r2, r2, r0, #4   // r2 = r2 + 4 * r0
C   IMAD  r3, r3, r0, #4
D   LD    r4, r1
    LD    r5, r2
```

4 * 32

C IMAD ...

D LD r4, r1

E LD r5, r2

F FADD r6, r4, r5

G ST r6, r3

insn * 32 → 4 * 32

A IF ID EX MEM WB
EX MEM WB
EX

32 → T0 in W0, T1 in W0

EX MEM WB

32

B. IF ID

C IF

32

n n+1 n+2 n+3 n+4 n+5 | n+6 n+7 ... 32

IF ID EX1 EX2 EX3 EX4 EX5 MEM WB

F EX1 E2 E3 E4 E5 MEM WB

E1

C[0] EX MEM WB

G IF ID

FD: 32, ALU: 32

IF ID EX MEM WB T0~31 of W0

FADD IF ID EX1 EX2 EX3 EX4 E5 MEM WB

st IF ID S S S S GX MEM WB

↓ more than 32 elements

RF: SM: 64 kB ⇒ 4B ⇒ 16 k registers.

↓

of warps. kernel → register usage

of TBs